

Лабораторная работа № 4

Тема: Разделение маршрутов Express-приложения с помощью Router

Цель: Научиться использовать `express.Router()`, выносить маршруты в отдельные модули, экспортировать и импортировать Router, подключать маршрутизаторы через `app.use()` и организовывать структуру серверного проекта по ресурсам.

Практическое задание

Продолжить разработку проекта **Internet Shop API**.

На предыдущих занятиях маршруты товаров, категорий и заказов находились непосредственно в `app.js`.

Необходимо выполнить **рефакторинг проекта**: вынести маршруты в отдельные файлы, не изменяя существующие URL API.

После выполнения работы `app.js` должен отвечать преимущественно за:

- создание Express-приложения;
- подключение общих middleware;
- подключение routers;
- запуск сервера.

Последовательность выполнения

1. Откройте существующий проект `express-shop`.

Убедитесь, что он запускается:

```
npm start
```

Перед изменениями проверьте основные endpoints:

```
GET /api/products
```

```
GET /api/products/1
```

```
GET /api/categories
```

```
GET /api/orders
```

2. Создайте каталог `routes`.

Структура должна стать такой:

```
express-shop/  
├── routes/  
├── app.js  
├── package.json  
└── package-lock.json
```

3. Создайте файл `routes/productRoutes.js`.

Подключите Express:

```
import express from "express";
```

Создайте Router:

```
const router = express.Router();
```

Перенесите в этот файл маршруты товаров.

Минимально должны присутствовать:

```
GET /
GET /:id
POST /
PATCH /:id
DELETE /:id
```

Обратите внимание: внутри Router больше не используется полный путь:

```
/api/products
```

Вместо него используются локальные пути:

```
/
и:
/:id
```

4. Экспортируйте product Router.

В конце productRoutes.js:

```
export default router;
```

5. Подключите productRoutes в app.js.

Импортируйте Router:

```
import productRoutes from "../routes/productRoutes.js";
```

Подключите его:

```
app.use("/api/products", productRoutes);
```

После этого запрос:

```
GET /api/products
```

должен попадать в:

```
router.get("/")
```

а:

```
GET /api/products/5
```

в:

```
router.get("/:id")
```

6. Проверьте работу маршрутов товаров.

Проверьте HTTP-клиентом:

```
GET /api/products
```

```
GET /api/products/1
```

```
POST /api/products
```

```
PATCH /api/products/1
```

```
DELETE /api/products/1
```

URL должны остаться такими же, как до рефакторинга.

7. Создайте categoryRoutes.js.

Создайте отдельный Router для категорий.

Минимальные маршруты:

```
GET /
```

```
POST /
```

При необходимости можете добавить:

```
GET /:id
PATCH /:id
DELETE /:id
Экспортируйте Router:
export default router;
```

8. Подключите Router категорий.

В app.js:

```
import categoryRoutes from "../routes/categoryRoutes.js";
```

Подключение:

```
app.use("/api/categories", categoryRoutes);
```

Проверьте:

```
GET /api/categories
POST /api/categories
```

9. Создайте orderRoutes.js.

Создайте Router для заказов.

Минимальные маршруты:

```
GET /
GET /:id
POST /
PATCH /:id
```

Экспортируйте Router и подключите его в app.js:

```
app.use("/api/orders", orderRoutes);
```

10. Приведите app.js к компактному виду.

В нём должны остаться:

```
import express from "express";

import productRoutes from "../routes/productRoutes.js";
import categoryRoutes from "../routes/categoryRoutes.js";
import orderRoutes from "../routes/orderRoutes.js";

const app = express();
const PORT = 3000;

app.use(express.json());

app.use("/api/products", productRoutes);
app.use("/api/categories", categoryRoutes);
app.use("/api/orders", orderRoutes);

app.listen(PORT, () => {
  console.log(`Server: http://localhost:${PORT}`);
});
```

Допускается наличие нескольких общих маршрутов, например:

```
GET /
```

GET /api

11. Проверьте итоговую структуру проекта.

Минимальный вариант:

```
express-shop/  
├── routes/  
│   ├── productRoutes.js  
│   ├── categoryRoutes.js  
│   └── orderRoutes.js  
├── app.js  
├── package.json  
└── package-lock.json
```

12. Проверьте формирование полного URL.

Для каждого примера определите итоговый endpoint.

Дано:

```
app.use("/api/products", productRoutes);
```

и:

```
router.get("/");
```

Результат:

GET /api/products

Дано:

```
router.get("/:id");
```

Результат:

GET /api/products/:id

Дано:

```
router.patch("/:id");
```

Результат:

PATCH /api/products/:id

13. Проверьте все ресурсы HTTP-клиентом.

Минимальный набор:

GET /api/products

GET /api/products/1

POST /api/products

PATCH /api/products/1

DELETE /api/products/1

GET /api/categories

POST /api/categories

GET /api/orders

GET /api/orders/1

POST /api/orders

PATCH /api/orders/1

Требования к результату

Проект должен:

- содержать каталог routes;
- содержать минимум три отдельных Router;
- использовать `express.Router()`;
- экспортировать каждый Router через `export default`;
- импортировать routers в `app.js`;
- подключать их через `app.use()`;
- использовать отдельный URL-префикс для каждого ресурса;
- сохранять существующие URL API после рефакторинга;
- не содержать маршруты товаров, категорий и заказов непосредственно в `app.js`;
- запускать только один HTTP-сервер через `app.listen()`.